

Alloy: Transparent Proxy-Based Coupling Between Stream Processing and Storage

Guillaume Rosinosky
IMT Atlantique, Inria, LS2N
Nantes, France
guillaume.rosinosky@inria.fr

Donatien Schmitz
ICTEAM, UCLouvain
Louvain-la-Neuve, Belgium
donatien.schmitz@uclouvain.be

Etienne Rivière
ICTEAM, UCLouvain
Louvain-la-Neuve, Belgium
etienne.riviere@uclouvain.be

Abstract

Stream storage and processing workflows support a wide range of applications, from enterprise data lakes to IoT analytics. Streams are ingested and stored in broker clusters, e.g., using Apache Kafka, while processing engines like Apache Flink execute queries. In such a setup, however, streams are read, sent, and (de)serialized in full between storage and processing, even when only a subset is relevant, leading to inefficiencies in networking and computation.

We present Alloy, a novel approach for reducing the cost of coupling stream storage and processing. Alloy filters data on storage nodes and sends only what is useful for a specific query to the processing cluster. A distinctive feature of Alloy is its transparency: the data plane uses WebAssembly plugins deployed atop the Envoy proxy, interposing between Apache Flink and Apache Kafka to perform selection and projection without modifying either system. The Alloy control plane analyzes queries as part of the regular Flink submission and configures proxies accordingly. Evaluation using microbenchmarks and Nexmark shows that Alloy drastically reduces network usage while maintaining or reducing CPU utilization, with minimal latency impact.

CCS Concepts

• **Computer systems organization** → **Data flow architectures**; • **General and reference** → **Performance**; • **Information systems** → **Data streams**.

Keywords

Stream processing, Service Mesh, Proxy, Flink, Kafka, Envoy

ACM Reference Format:

Guillaume Rosinosky, Donatien Schmitz, and Etienne Rivière. 2026. Alloy: Transparent Proxy-Based Coupling Between Stream Processing and Storage. In *The 20th ACM International Conference*

on Distributed and Event-based Systems (DEBS '26), June 23–26, 2026, Lisbon, Portugal. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3809481.3812609>

1 Introduction

Many industrial sectors rely on processing large volumes of *event streams*. Several systems support the ingestion, storage, and processing of stream data, both historical and as it is received in real time [36]. Among these systems, two families, *stream storage* and *stream processing*, complement each other to support stream workflows. Stream storage focuses on the reception, persistence, and replay of events, while stream processing supports complex queries combining multiple sources of historical and real-time data. Examples of stream storage platforms include Apache Kafka [46], LiftBridge [49], NATS/JetStream [52], Pulsar [43], RabbitMQ [55], and RedPanda [57]. Examples of stream processing platforms include Apache Flink [29], Storm [65], Spark Streaming [71], as well as research proposals such as Cameo [69] or WASP [41]. The combination of Kafka and Flink, arguably the most popular in the industry to support stream workflows [1, 2, 44], is our focus in this paper.

Stream storage with Kafka and stream processing with Flink are designed for cluster or cloud deployments, but with different deployment constraints. Kafka balances storage load across a fleet of *brokers* and uses replication to mask faults. Brokers are deployed for the long term and favor machines with ample, low-cost block storage (e.g., HDDs) [17]. In contrast, Flink deployments use an elastic-scaling fleet of Task Manager nodes, orchestrated by a permanent Job Manager node. Task Manager nodes' requirements are primarily CPU and memory [16]. They do not require replication for fault tolerance but rather rely on snapshot and restore mechanisms [28]; a cost-effective approach can even be to deploy them on cheap but transient spot instance cloud resources [21]. In some cases, finally, stream storage and processing may happen in different locations (data centers) [23, 24, 50, 51, 72].

Motivation. Separating stream storage and processing into different systems enables adherence to their distinct deployment requirements: a stable fleet of storage-focused nodes on



This work is licensed under a Creative Commons Attribution 4.0 International License.

DEBS '26, Lisbon, Portugal

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2693-4/2026/06

<https://doi.org/10.1145/3809481.3812609>

the one hand and an elastically scaling fleet of CPU/memory-focused instances on the other. However, it requires efficiently *coupling* the two systems, i.e., allowing stream processing to fetch and process data from stream storage. A key observation is that, in practice, stream queries require only a subset of the data of the streams they process. Our analysis of 129 queries in a data lake system combining Kafka and Flink shows, for instance, that more than 50% of the queries use less than 50% of the data in streams they consume [62]. Kafka, nevertheless, is oblivious to the nature of queries and only serves data at the granularity of entire streams. This leads to coupling inefficiency. Indeed, while streams of events are sent in full over the network by Kafka, in Flink's end, these events are deserialized and reified as Java objects only to be immediately dropped or trimmed of unnecessary fields. This results in wasted interconnect bandwidth between the clusters and additional CPU utilization at the Flink level.

A natural idea for improving the coupling between storage and processing is to allow data filtering at the source, i.e., on the Kafka cluster, and to provide a curated stream of events containing only data of interest to a specific query. Implementing this idea, however, is far from trivial. Extending Kafka to support on-the-fly processing of source stream data, as proposed by Query-able Kafka [33], requires deep, hard-to-maintain modifications to the system, and does not solve the problem of extracting filtering needs from queries in the first place. While Kafka supports limited querying with Kafka Streams [15, 27], using these capabilities to create new, curated streams for a specific query comes at a high cost. Not only must a dedicated node on the storage side be allocated to support the query, but the resulting curated streams must also be stored on disk, incurring significant transformation and storage costs, particularly for streams with large volumes of historical data.

Contribution. We propose Alloy, an efficient and integrated approach to improve the coupling between stream storage and processing using Kafka and Flink, enabling near-source data curation. In contrast to past proposals [33], Alloy does not require modifications to either system nor the materialization of new topics on disk [15, 27]. At its core, Alloy leverages the concept of sidecar *proxies*, as popularized in cloud-native systems under the *service mesh* model [48]. Under this model, proxies such as Envoy [3], Apache Traffic Server [8], or Nginx [6] are deployed alongside application components in the cloud. They help address system-level concerns such as load balancing, observability, and secure communication [48, 61], or even offer interoperability layers between legacy services [35].

In Alloy, dedicated proxies deployed alongside Kafka brokers transparently intercept connection and data requests from Flink. Alloy proxies efficiently handle selection and

projection requests on stream data read from Kafka, and provide Flink queries with curated streams containing only the information of interest. This, in turn, allows simplifying queries before submitting them to the Flink cluster. Alloy proxies are built as WebAssembly modules for portability and isolation, over the proxy-wasm generic proxy Application Binary Interface (ABI) [54]. We deploy them over the popular Envoy proxy [3].

In addition to proxies, which form its data plane, Alloy proposes an integrated control plane that intercepts the submission of generic SQL queries under the popular TableAPI model to Flink. This control plane transparently analyzes submitted queries, leveraging Apache Calcite [26], and extracts selection and projection operations. It then sets up proxies and sends the simplified query to Flink, all without any user involvement.

The design and implementation of Alloy target ease of maintenance and limit vendor lock-in. Our use of WebAssembly and the proxy-wasm ABI retains the flexibility to target other compatible proxies [4, 6, 8]. Furthermore, the Kafka protocol the proxies intercept is mature and supported as the *de facto* standard by other similar services, including RedPanda [57], AutoMQ [10], WarpStream [19], or even PaaS cloud services such as Azure Event Hubs [9].

We evaluate Alloy on a 10-node cluster, using microbenchmarks and the standard Nexmark benchmark [66]. Our results show that the impact of Alloy proxies on latency is very small (i.e., a few milliseconds). CPU utilization at these proxies depends on the nature of selection and projection operations required by a query, but often matches or is lower than the corresponding gain at the query side. We further show that Alloy can drastically reduce network consumption while keeping overall CPU utilization similar to, or lower than, when using a direct coupling of Kafka and Flink.

The source code of Alloy and instructions to reproduce our experiments are available under open source and open science principles [60].

Outline. We give the necessary background on stream storage and processing with Kafka and Flink, and a primer on service meshes and sidecar proxies in Section 2. We refine our problem statement with example Nexmark query measurements in Section 3. We provide an overview of Alloy in Section 4, before detailing its data plane (proxies) in Section 5 and its control plane in Section 6. We present our evaluation in Section 7. Finally, we present related work in Section 8 and conclude in Section 9.

2 Background

We review the stream storage and processing concepts that are useful for detailing our motivation and contributions. Moreover, we detail the principles of proxies and service

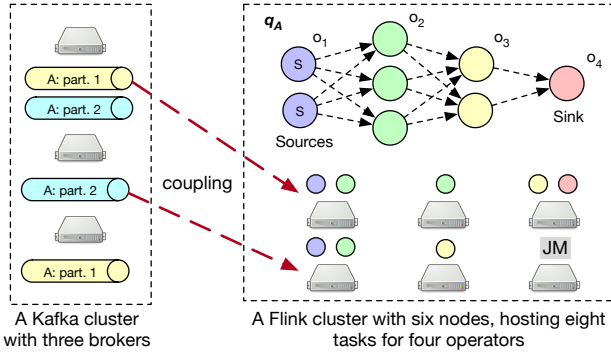


Figure 1: Example coupling of a Kafka cluster with three brokers storing a single topic A with two partitions and a replication of two, with a Flink cluster of six nodes (one Job Manager, five Task Managers) running a query q_A with eight tasks.

meshes as used in modern, cloud-native platforms, which play a primary role in our solution.

2.1 Stream storage and Apache Kafka

A stream storage platform ingests, stores, and replays streams of events with strong durability and availability guarantees.

Apache Kafka is a stream storage platform with strong commercial and community support. Its API and client-server interaction protocols have become *de facto* standards, supported by several standalone and managed stream storage services [9, 10, 19, 57].

A Kafka cluster is formed of several *broker* nodes, as shown on the left side of Figure 1. Event streams are stored as distinct named *topics*. Each topic is split into one or several *partitions*. Each partition is replicated across several brokers, with a replication factor (typically, 3) for fault tolerance. New data events (records in Kafka terminology) sent by producers are dispatched to partitions either randomly or based on a hash of a user-specific identifier. A schema registry [18] can be used to store schema information (e.g., in Avro or JSON) alongside topic information, but broker operations are otherwise schema-agnostic.

Interactions with a Kafka cluster allow a client to fetch the list of partitions for a topic, and then fetch events at a specific offset in one of these partitions. Kafka guarantees in-order delivery within a partition, but not across partitions. Two important parameters enable a compromise between efficiency (through batching) and latency (via fast responses). The `fetch.min.bytes` option gives the minimum amount of event data the server should return for a fetch request. A value of one Byte means that fetch requests are answered as soon as data is available; a higher value improves throughput at the cost of additional latency. This additional latency can

be capped using the `fetch.max.wait.ms` parameter, which gives a maximal waiting time before a response to a fetch request is given.

2.2 Stream processing and Apache Flink

Stream processing operates queries, typically expressed as a graph of logical *operators* [36, 39], as shown with query q_A on the right-hand side of Figure 1. In Flink, logical operators include stateless operations such as *maps* and *filters*, and stateful aggregation operators such as *groupby-aggregate* over diversely defined event and time windows [67]. Besides unary operators, *join* and *union* stateful operators consume and combine multiple streams. While this graph can be expressed programmatically, it is increasingly often compiled from a high-level declarative statement in a variant of SQL [68], which is easier to use and fits the common case of processing tabular data.

The execution of a query in Flink maps logical operators to an elastically scalable set of *tasks* orchestrated by a Job Manager (JM), with the stream content for an operator split across its tasks for parallel processing [30, 58]. Each task is handled by a single-threaded *task slot* on a Task Manager (TM) node. Higher input rates for a query lead to scale-out operations to cope with increased CPU load and state storage requirements; hence, more Task Managers are spawned in the cluster, e.g., by the associated Kubernetes operator [5].

Specific *source* operators are in charge of reading and feeding stream data to the query, while *sink* operators write results to an external service. The Kafka source operator [7] subscribes to a topic specified at query instantiation time, and can fetch the stream from specific offsets of its assigned partitions. The parallelism of source operators is generally fixed [42], and the list of partitions returned by Kafka when initially queried about the topic is split across its tasks (i.e., each source operator task handles one or more partitions, and each partition is handled by a single task). Source operator tasks use *deserializers* objects specific to the event encoding (e.g., Avro or JSON) to transform events received from Kafka into Java objects, according to a schema. Depending on the complexity of the events, this deserialization can be particularly CPU-intensive.

2.3 Service meshes and sidecar proxies

Deploying applications in the cloud often requires handling multiple, complex system-level aspects, such as traffic management, load balancing, traffic monitoring, telemetry, or secure communication. Adding support for such aspects in application code (e.g., application microservices or support frameworks such as data processing engines) introduces code

redundancy and limits coordinated management across applications. A service mesh such as Istio [11] addresses these issues and simplifies the deployment of complex, cloud-native workloads. It does so by forming an additional, application-level network between application components and system services. This network typically involves *sidecar proxies* deployed alongside application components, and enforces system-level aspects without modifying the supported applications.

The Envoy proxy [3] is one of the most advanced and popular sidecar proxies. It intercepts communication to and from supported applications and can modify application-level traffic (e.g., to enforce encryption in zero-trust architectures) or network-level traffic (e.g., to implement a specific load-balancing policy).

Envoy increasingly supports flexible Kafka deployments. The experimental Kafka Broker filter [12] supports Kafka-specific monitoring but also limited forms of filtering between brokers (e.g., to separate produce and fetch requests to different instances), or assists with transparent routing in geo-distributed broker networks [45]. The Kafka Mesh filter [14] is another recent Envoy proxy extension that provides a load-balancing facade for Kafka clusters. While Flink does not currently have specific Envoy filters, generic ones are often used in its deployments for telemetry or to transparently secure communications.

3 Motivation and problem statement

In this section, we refine the motivation and problem statement introduced in Section 1.

Our objective is to reduce the cost of coupling a query running on a Flink cluster and the access to its input data in a Kafka cluster. We wish to reduce network consumption between the two clusters by sending only the necessary data, i.e., avoiding sending data that query operators will never use. In addition, we hope to save CPU and memory resources on the Flink side and, as a result, avoid unnecessary scale-out operations. Our rationale is that sending only the necessary data between the Kafka and Flink clusters relieves source operators of the need to deserialize data and create unnecessary objects. It also allows removing *filter* and *map* operators from the operator graph when their sole purpose is to discard unnecessary information, thereby reducing the complexity of the query graph and the number of required task slots.

In complement to the observations we made in our previous work [62], as mentioned in Section 1, we show in Figure 2 an observation of the six most popular queries of Nexmark [66], a *de facto* standard benchmark for stream processing systems. Nexmark simulates an auction system with a set of representative queries. We observe that all queries

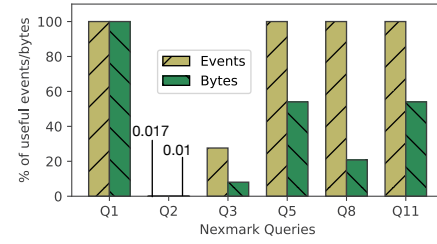


Figure 2: Percentage of useful events and bytes for six popular queries of the Nexmark benchmark [66].

but Q1 read only a subset of the event data (i.e., fields in the events’ schemas), and sometimes a very small fraction of it: less than 10% for Q3, an incremental join preceded by two filters, and even less than 0.01% for Q2, a stateless filter and projection query. While four queries read from all events in the input stream, this is not the case for Q2 (a filter operator selecting bids based on an identifier) and Q3.

We aim to support initial filtering (pre-selection and pre-projection) of stream data near the source, i.e., directly on the nodes hosting Kafka brokers handling target topic partitions. We further refine our non-functional objectives as follows:

- We wish to enable this initial filtering without changing the code of either Flink or Kafka. Such changes are indeed hard to maintain and persist across software versions. Introducing them could also tie the evolution of code in one system to that of the other, breaking the clean separation that using the stable Kafka API enables.
- The initial filtering should not require additional storage space on Kafka brokers. Generating and storing pre-filtered data in advance of query execution lacks flexibility and risks wasted effort. Furthermore, generating new topics upon query execution may lead to unacceptable delays in query establishment, as well as increased CPU and disk usage. Filtering must, therefore, be an “on the fly” operation.
- The user experience when submitting queries for execution should not be affected, and no additional configuration or setup should be required.

We make the following assumptions, all of which are reasonable for a cloud-native deployment of a stream workflow used by data analysts (e.g., in an enterprise *data lake*):

- The schemas used by events received and stored in Kafka are registered in a schema registry, or statically known.
- Users submit their stream processing queries as SQL statements that are compiled to Flink operators graphs thanks to the SQL-compatible TableAPI Flink interface.
- Kafka and Flink are deployed in a containerized environment (e.g., using Kubernetes) with a service mesh. Envoy proxies are deployed alongside Kafka broker nodes. We

do not require a specific Alloy code to be pre-deployed on these Envoy proxies, but we assume they support the runtime instantiation of proxy functions via plugins.

- The Kafka cluster configuration (number and location of brokers) is stable during a query execution (i.e., any elastic scaling of Kafka, and subsequent re-allocation of partitions, happens outside of query execution).

4 Overview

Alloy is a new solution to improve the coupling between Kafka-based stream storage and Flink-based stream processing. It consists of two main components.

Alloy proxies, detailed in Section 5, form the *data plane*. Their role is to intercept communication between the Kafka client (i.e., the source operator in the Flink query) and the Kafka broker. In doing so, the Alloy proxies return only information relevant to the query, based on an initial configuration that specifies the necessary filtering and projection of events in the stream. These proxies are implemented with high performance and portability as first-class objectives.

The configuration of the proxies, i.e., setting the necessary selection and projection operations for a given query, is handled by the Alloy *control plane*, detailed in Section 6. As the Alloy proxies intercept communication between Flink and Kafka, the control plane is interposed between the client that submits a query and the Flink cluster. It extracts from the graph representation of the compiled query selection and projection operations for events and fields in these events that are not necessary to the query. It sets up the Alloy proxies at the appropriate broker(s), strips the compiled graph representation of the query of unnecessary operators, and submits it to Flink for execution.

5 Data plane: Alloy proxies

We present the design and implementation of the Alloy proxies. We first detail how we intercept and modify the messages exchanged between the client source operator and the Kafka cluster. Then, we present our implementation and how we enforce performance and portability.

5.1 Proxying the Kafka protocol

Figure 3 illustrates how Alloy proxies intercept communications between the distant Flink cluster (i.e., emitted by source operators acting as Kafka clients) and the co-located Kafka broker. A proxy can forward the received packets to and from the broker, or intercept and modify them. The Alloy proxy understands the Kafka protocol messages exchanged between a client and a broker and can modify their content. The Kafka protocol can be split into two phases.

The first phase is a *setup* phase, in which the source operator requests information about the target topic, its partitions,

and the current offsets for reading these partitions. These operations are illustrated in the upper areas of Figure 3. Alloy does not modify setup messages between the source operator and the broker, in any direction (i.e., it acts as a *pass-through* proxy). It only verifies that the API version returned in the first interaction matches one it supports.

The second phase is the *fetching* phase, and consists of repeated fetch-and-reply exchanges between the source operator and the broker. Fetch messages sent by the source operator specify the two parameters discussed in Section 2, i.e., `fetch.min.bytes` and `fetch.max.wait.ms`, in addition to the main request parameters (topic, partition, and read offset). The broker responds with event data either when there is enough of it to fill a buffer of `fetch.min.bytes` Bytes, or when the time since the reception of the request exceeds `fetch.max.wait.ms` milliseconds, as illustrated on the left side of Figure 3. Fetch requests sent by the source operator are forwarded *as is* to the broker. However, responses in the form of Events messages are intercepted. The operations performed depend on whether selection and/or projection actions have been set up. Selection removes events that fail a check on one or more criteria, applying to specified field(s) in the event schema. Checks are expressed as regular expressions over the string translation of a field value. In the example of Figure 3, only events whose color field is “green” are sent to the query. Projection selects fields of interest from the event schema and discards the others. In the example of Figure 3, we can assume that the query is only interested in the shape and color of the events, but not in the field that corresponds to some internal value (here, represented by a letter A, B, C, ...); this latter field is, therefore, not projected in returned events. Events response contains a list of events with each corresponding offset in the partition. The offset of the last event is used in the next Fetch request to read the follow-up events (Kafka clients can support non-contiguous offsets). The last event of the returned batch may be discarded by selection, including in the case where no event is returned at all in Figure 3, right, second interaction. In this case, the proxy still includes a dummy (null) value for the last offset received from the broker. This event will be interpreted by any compliant Kafka client as a “tombstone” record, as planned in the protocol to support deletions [13]. Returning this null record with the correct offset allows the client source operator to request the next batch of events as in the non-proxied case.

Interactions between a source operator and a broker via the proxy respect the `fetch.max.wait.ms` parameter, modulo a small (and often negligible) additional time spent processing the response by the proxy in comparison to the vanilla case (i.e., at only the broker). We choose, however, not to enforce the `fetch.min.bytes` constraint at the proxy level: as shown in Figure 3, on the right side, first fetch query,

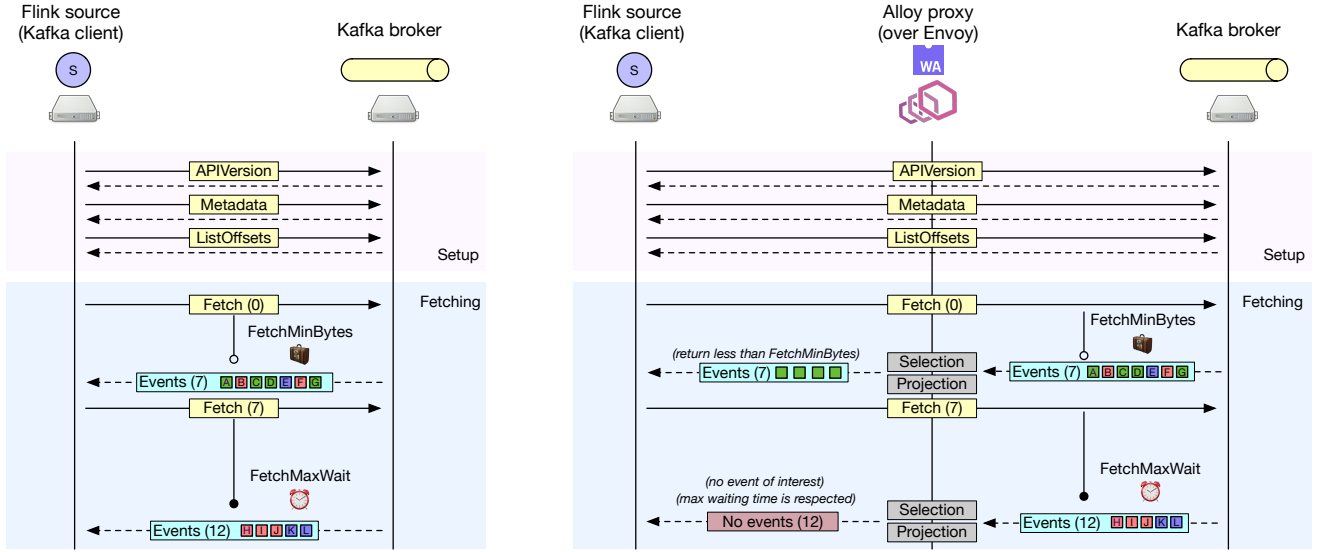


Figure 3: Direct interactions between the Flink source operator and the Kafka broker (left), and interactions via the Alloy proxy (right). Filtering is symbolized by a query that requires only green events (selection) and does not require the letter label field (projection). The Alloy proxy forwards connection setup messages, and intercepts response messages, applying selection and projection as necessary while respecting time constraints set by the source operator. Numbers in parentheses in Fetch and Events messages are the last read offsets in the topic.

the client may receive less than the requested volume of data (because it receives only a subset of the data of interest from the volume it initially requested). Respecting this constraint would require buffering events of interest at the proxy, breaking the simple stateless design and necessitating recovery mechanisms in case of a fault (in the stateless model, a failed proxy only leads the client to resend the request and read from the same offset).

5.2 Proxy implementation

We implement the Alloy proxy over the Envoy proxy [3], discussed in Section 2. Our implementation aims for performance and resource efficiency, as well as portability and ease of deployment.

The Alloy proxy focuses on the consumer API of Kafka. Messages may span multiple TCP packets, and a packet may contain (parts of) different messages. The Alloy proxy must, therefore, identify and reconstruct messages. Each message starts with a 4-Byte header indicating its length. Messages are collected until they are available in full and ready for processing. The main source of CPU consumption by the proxy is in deserializing received events and in serializing them back to return to the client. Memory copy operations should also be avoided for performance reasons. With this intent, the Alloy proxy is implemented in Rust and designed to avoid these operations whenever possible. Deserialization of the body of the message is used only when selection or

projection criteria are set; otherwise, the proxy acts as a *pass-through* and forwards the unmodified traffic. Serialization is used only when projection criteria are used. Otherwise, traffic is reconstructed from the original buffered data by reusing the binary ranges corresponding to the non-filtered events. As we will show in our microbenchmark evaluation in Section 7, the proxy’s latency impact is minimal.

We also wish to be able to deploy the Alloy proxies (and their evolutions) rapidly, without re-deploying Envoy proxies themselves, which may be in place and support other, critical operations in the cluster, such as security or monitoring. Furthermore, we wish to deploy Alloy proxies across other proxy and service mesh technologies. We achieve these two objectives by compiling the Alloy proxy code into WebAssembly as a *plugin* for the proxy-wasm interface [54]. WebAssembly is a portable, low-level binary instruction format designed to run code at near-native speed in a secure, sandboxed environment. It enables programs written in languages like C, C++, and Rust to run efficiently in web browsers and other runtimes beyond the web. The proxy-wasm interface is supported by a variety of proxies and service meshes, such as Apache Traffic Server [8], Nginx [6], and the MOSN proxy (Modular Open Smart Network [4]). In contrast to the classical Envoy features (named “filters”), using a proxy-wasm plugin provides better isolation, reduced vendor lock-in, and greater deployment flexibility, at the cost of a small runtime overhead due to WebAssembly.

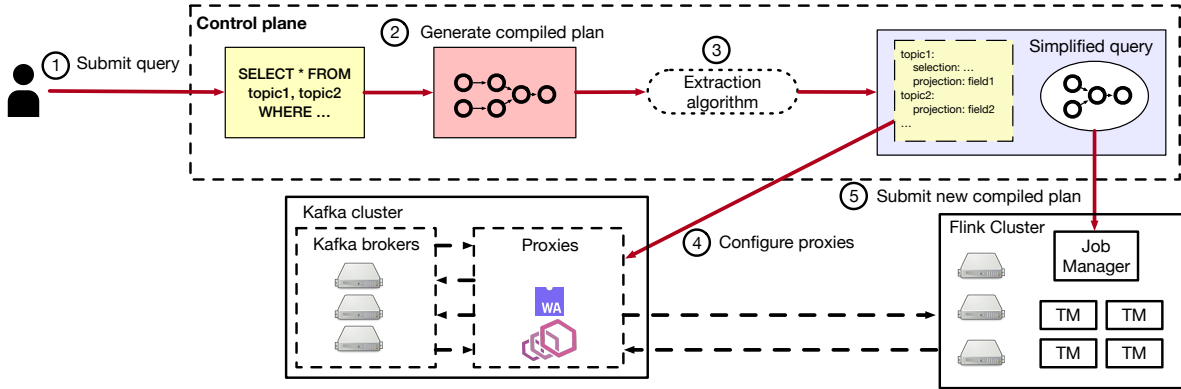


Figure 4: Complete submission and execution workflow through the Alloy control plane.

6 Alloy control plane

The Alloy proxies must be configured according to the queries. To this end, the Alloy control plane plays three principal roles, as illustrated by Figure 4. It offers the same interface (API) as Flink for submitting queries written in SQL. It intercepts a submitted query ① and proceeds in four steps. First, it generates the graph representation of the query operators, called a *compiled plan* ②. Then, it extracts from this plan and the events schema the selection and projection operations that restrict the input flow to exactly what is necessary for the query ③. On this basis, it can produce a new, *simplified* version of the query, i.e., the graph without the operators that were generated in the compiled plan to filter out unnecessary data, since this task is now performed on the proxies. Then, the query deployment proceeds in two phases. The control plane identifies the topics and their partitions required to run the query by querying the Kafka cluster (any broker can return the mapping between topics, partitions, and brokers). The proxies associated with the brokers in charge of these partitions are set up, deploying the WebAssembly module over Envoy if necessary, and configured with the extracted selection and projection criteria ④. Finally, the simplified query is sent to the Flink cluster ⑤. The source operator’s connections to the designed brokers are intercepted by the proxies, and query execution can begin. In what follows, we focus on the transformation operations (② and ③).

The identification of selection and projection occurs using Flink’s intermediate query representation as a graph of operators, i.e., the result of compiling the SQL query. The Alloy control plane leverages the fact that this compilation in Flink builds on Apache Calcite [26], an open-source dynamic data management framework that provides query parsing, validation, optimization, and SQL planning capabilities. The SQL query is transformed using Calcite into a graph-based representation, returned as a JSON file, where vertices are operators (including source and sink operators), and edges

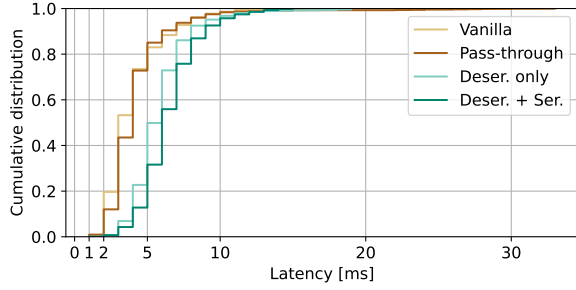
represent event streams between them. When compiling a query from SQL, Flink instructs Calcite to identify unnecessary fields for its operators and add a specific projection operator right after the corresponding source operator. Similarly, selection operators are typically placed right after the source operator, corresponding to “WHERE” statements in the query. The Alloy control planes identify these two operators for each source stream (topic) used by the query in the graph returned by Calcite. For projection operators, it determines which fields from the schema are *not* used, i.e., the complement of those kept by the projection operator in the event schema. For selection operators, it determines if the selection criteria can be supported by the Alloy proxy (i.e., as regular expressions) and, if so, extracts them. Finally, the graph representation is stripped of the now-unnecessary operators, and the simplified query is generated directly as a graph JSON representation that is submitted to the Flink cluster for execution.

7 Evaluation

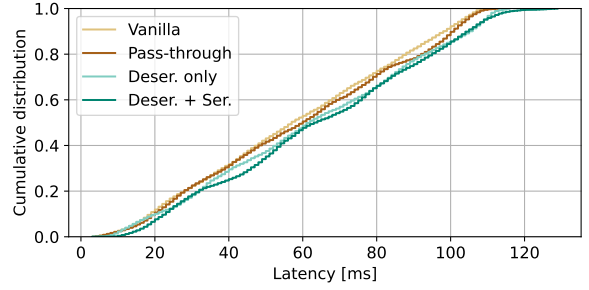
We evaluate Alloy on a representative testbed, using microbenchmarks and the Nexmark benchmark [66]. We aim to answer the following research questions (RQ):

- RQ1 What is the cost of an Alloy proxy in terms of resource utilization, when using different configuration modes (using selection only, projection only, both, or acting as a pass-through)?
- RQ2 Is Alloy able to reduce the use of the network compared to the vanilla case of coupling Kafka and Flink directly?
- RQ3 What is the CPU impact at the Kafka side of using the Alloy proxy, and can this cost compensate the associated savings at the Flink side?

We use two primary metrics to reflect the performance of the coupling between the Kafka and the Flink clusters. The first is the volume of data exchanged between the two clusters. The second is the cumulative CPU utilization on the



(a) fetch.min.bytes = 1 B.



(b) fetch.min.bytes = 1 MB.

Figure 5: Intrinsic latency of the Alloy proxy in different modes, for two values of the `fetch.min.bytes` parameter.

side of the Kafka cluster (with and without the Alloy proxies) and on the side of the Flink cluster running the query, and its distribution between the different components of the system. Alloy aims to lower the first metric and contain inflation in the second (i.e., to keep the proxies' overhead at a similar or lower level to what is gained on the Flink side). Memory consumption by the proxies is also negligible for practical values of the `fetch.min.bytes` parameters (i.e., a few MBs), and never a bottleneck in practice. Last, Alloy proxies do not require disk storage. Therefore, we do not report memory and I/O-related metrics. The Alloy proxies were never found to be a throughput bottleneck on our brokers, where reading performance is primarily capped by the SSD.

We also consider the latency of data-plane operations, i.e., from the read of data from Kafka until the data reaches the Flink cluster. Control plane operations as part of a query submission always take below 1 second, an extra duration that is negligible compared to the regular query submission and bootstrap time in a Flink cluster, which is on the order of a dozen seconds.

In what follows, we start by detailing our experimental setup. Then, we present microbenchmarks that highlight the intrinsic costs of Alloy proxies in terms of CPU utilization and operation latency, answering RQ1. Finally, we review the performance of Nexmark queries to answer RQ2 and RQ3.

7.1 Experimental setup

Our testbed, part of the Grid'5000 grid [25], comprises 10 nodes. Each is equipped with an 18-core Intel Xeon Gold 5220 CPU, 96 GB of RAM, and a 480 GB SSD. All nodes are connected by a 10 Gbps local network and run Debian 11. We use versions 1.16.0 of Flink, 1.29 of Envoy, and 3.4.0 of Kafka.

We use a level of parallelism for the Kafka cluster ranging from 1 to 4 brokers, each deployed on a separate node. Each broker hosts a single, non-replicated partition of each

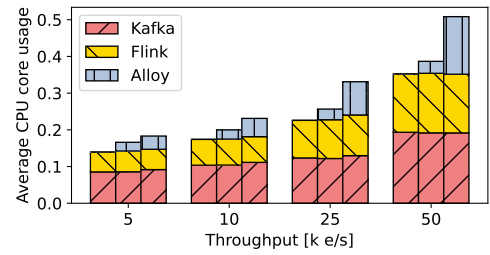


Figure 6: Cumulative, worst-case CPU utilization of using the Alloy proxies with an *identity* query that uses all events and all fields in these events. For each throughput (in events/s), the three bars correspond to (1) the vanilla mode with only Kafka and Flink, (2) the pass-through mode, and (3) the Deser. + Ser. mode.

topic. Another node hosts a specific job that acts as an event generator, pushing events to Kafka topics at a constant, configurable rate. This same host is used for Prometheus metrics collection. The remaining five nodes are dedicated to Flink. One of these nodes hosts the Job Manager. The final four nodes host the Flink query. Task Managers are configured to receive 2 virtual CPUs and 2 GB of RAM each. The cluster configuration uses Docker Swarm. All experiments are scripted for reusability using Jupyter notebooks parameterized by Papermill, and are available with the Alloy source code [60].

7.2 Microbenchmarks

Our microbenchmarks aim to answer RQ1 and evaluate the intrinsic resource usage cost and performance impact of the Alloy proxies.

Latency. We start by evaluating the proxies' cost in terms of end-to-end latency. We consider the Kafka cluster and a simple Flink query that comprises only a client source operator. In the *vanilla* mode, there is no proxy, and the source operator connects directly to the broker. In the *pass-through*

mode, the proxy intercepts all messages. It determines their nature and size by consulting the first bytes of each message, but forwards them without accessing their content. In the *deserialization only* (Deser. only) mode, the proxy deserializes the message's events, corresponding to cases where a selection operation is necessary and a field in the event must be verified. Finally, in the *deserialization and serialization* (Deser. + Ser.) mode, the events are further re-serialized in the message before being sent back to the source operator.

We generate a constant rate of 1,000 events per second to a single broker instance. Each event contains 9 fields and is 500 B in size. The parallelism of the Flink query is limited to one task, for a unique operator. Figure 5 presents the latency between the creation of a message by the generator and its reception by the query, for two values of the `fetch.min.bytes` parameter. With the minimum value of 1 B, the accumulated stream data at the broker since the last Fetch is immediately returned to the query. With a value of 1 MB, the Fetch request is answered when 1 MB of data accumulated at the broker for this topic partition. Naturally, the latter configuration results in latency dominated by the buffering time at the broker, but it can support higher throughputs. In Figure 5(a), we observe that the proxy's impact on end-to-end latency in *pass-through* mode is minimal. The Deser. only mode incurs an additional 2 ms latency per message. Enabling serialization adds about 1 ms to that. A 3 ms latency increase is negligible in comparison to the typical latency of a Flink query processing, in particular in the common case where events must accumulate in a time window before leading to an output. When using a larger buffer size, as in Figure 5(b), we observe similar results, with a slightly higher latency increase due to the larger message size, but still small compared to the vanilla latency dominated by buffering times (i.e., the difference between the vanilla distribution of latencies and the ones involving Alloy proxy operations remains without a few ms, and is proportionnally negligible when compared to the natural latency of Kafka API operations with the same configuration).

Worst-case CPU usage. Our second microbenchmark measures the proxy's CPU usage in the worst case. The Flink cluster executes an identity query that receives and consumes all events from a topic and all fields in these events. The query is very simple and consists of a single source operator. As previously, we use a parallelism of 1 for both Kafka and the Flink query. We emphasize that, for such a query, the Alloy control plane would detect the absence of selection and projection operations and configure the proxies in *pass-through* mode. Nevertheless, to measure the CPU usage of serialization and deserialization, we also force the Deser. + Ser. mode. Figure 6 presents the results. We observe that the CPU cost of the Alloy proxy is less than 0.1 CPU core on

average in *pass-through* mode and remains stable across different throughputs. The cost of serializing and deserializing, however, is proportional to the volume of events to process. This suggests that, to compensate for the increase in CPU utilization on the Kafka side, running the simplified query on the Flink side, with its reduced data volume, must reduce CPU utilization and, at a minimum, compensate for, and at best overcome, the proxy overhead. This is what our next experiment investigates.

Potential for CPU usage reduction. In this third microbenchmark, we use the same setup as before, but change the Flink query to perform a selection, a projection, or both. The selection retains 20% of the events based on a filter applied to one of the 9 fields in the schema. The projection retains 1 of these 9 fields, corresponding to 1% of the Bytes of each event. Figure 7 compares the CPU utilization of Flink alone and Flink plus the Alloy proxy. For the sake of readability, we do not include the CPU utilization of Kafka in this figure, which remains constant across experiments (as can be seen in Figure 6). We observe that the impact of Alloy on overall CPU utilization grows with event volume. With low throughputs, performing selection or projection at the proxy yields overall similar CPU utilization, while performing both results in a small reduction. With higher throughputs, however, the gap between Flink-only query execution and the use of Alloy widens, and unsurprisingly more so when both selection and projection are used. We note, however, that the gap remains relatively small, indicating that some queries may see reduced CPU consumption across the two clusters, while others may not (yet, all queries should see drastically reduced network consumption).

Our microbenchmarks answer RQ1 by showing that, in addition to the expected natural decrease in network bandwidth, using Alloy and its proxies can yield a potential gain or a *status quo* in CPU utilization, with minimal impact on event latency.

7.3 Macrobenchmark: Nexmark queries

We evaluate the impact of Alloy on the performance of queries from the Nexmark benchmark [66], to answer RQ2 and RQ3.

Nexmark simulates an auction system and provides a set of representative queries for evaluating batch and stream processing systems. The Nexmark data generator we use generates data for each topic, i.e., auctions, bids, and persons, that the queries consume at a target rate. We presented in Section 3, Figure 2, the ratio of useful events and data in the Nexmark queries commonly used in stream processing engine evaluations [42, 59, 63].

In this section, we focus on four queries to avoid analyzing queries that are very similar in nature and behavior. Results

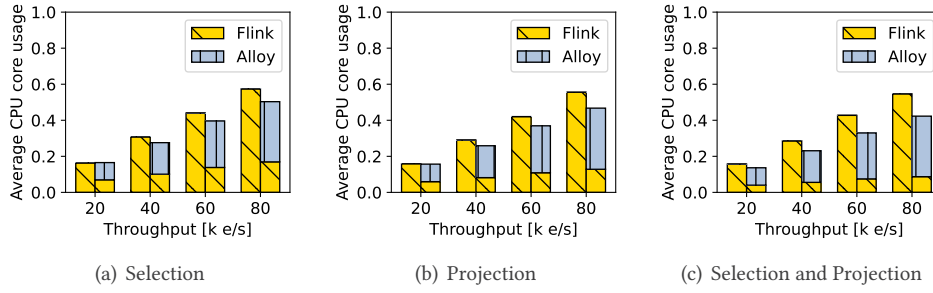


Figure 7: CPU utilization of Flink alone (left bars), and Flink plus the Alloy proxy (right bars), for three configurations: (1) the proxy only performs selection, (2) the proxy only performs projection, and (3) it performs both.

are presented in Figure 8. The effective throughput of a query depends on its nature, the workload, and the parallelism at which it runs. We consider parallelism levels of 1, 2, and 4 instances, applied both to Kafka brokers and to operators of the query (e.g., if a query with four operators runs at a parallelism level of 2, it will use two Kafka brokers and its operators will use a total of eight task slots in the Flink cluster). For each configuration, we target two throughput levels: a low one with moderate CPU utilization for the Flink TM, and a high one closer to the query’s Maximal Sustainable Throughput (MST), i.e., its saturation point. We use the following parameters in the source operators of the queries: `fetch.min.bytes` = 10 MB and `fetch.max.wait.ms` = 20 ms. As detailed in Section 5, this leads the source operators to fetch the same number of messages in the vanilla and Alloy configurations.

Query Q1 is a stateless, single-operator query that reads all events and all fields in these events. Naturally, we expect no gain from Alloy with this query, and only the overhead of the proxy in *pass-through* mode as we observed in our microbenchmarks. Results in Figure 8(a) confirm this: the network usage is the same from the vanilla configuration to Alloy, and the proxy CPU utilization does not result in a reduction in CPU utilization on the Flink side.

Query Q2 is also a single-operator, stateless query that, unlike Q1, uses only a small percentage of the topic’s data. Figure 8(b) shows that, in this case, Alloy can drastically reduce network utilization (numbers on the plot show values too small to be read on the bar plot). The CPU utilization of Flink is also significantly reduced, due to the much lower volume of events to process. Obviously, the selection and projection operations at the source and their associated (de)serialization requirements drive up the CPU consumption at the proxy, but this consumption is lower than what is gained at the Flink side, allowing to run this query even with minimal resources at the Flink side as long as enough spare CPU capacity is available on Kafka brokers.

Results for query Q3 are given by Figure 8(c). This query is a little more complex than the previous, as it performs

an unbounded incremental join and uses three operators, including a stateful one. The transformation of the query by the Alloy control plane reduces the query to only the stateful operator (in addition, of course, to the source and sink ones). We can observe, as for Q2, that the network utilization is drastically reduced across all configurations when using Alloy. CPU utilization at the Flink side is also reduced but not as significantly as for Q2, because the fraction of data Q3 uses is higher, but also because the CPU cost of the stateful join operator dominates. At low throughputs, the proxies’ CPU utilization is higher than the gain achieved in the Flink cluster. At higher throughputs, however, the situation flips, and the overall CPU utilization decreases.

Finally, we present results for query Q8 in Figure 8(d). Query Q8 implements a stateful tumbling window join. In all cases, Alloy significantly reduces network consumption between the storage and processing clusters. The CPU utilization is, however, slightly higher when using proxies than in the vanilla case. This overhead results from the low input rate compared to other queries. Indeed, we showed in Figure 7 that Alloy achieves savings starting at 40k e/s: Alloy has an higher overhead at lower rates. We observed similar results for queries Q5 and Q11, but omit their detailed results due to space constraints.

We can conclude this evaluation using the Nexmark query by answering the remaining two research questions, RQ2 and RQ3. For RQ2, we saw that Alloy can drastically reduce network consumption between the Kafka cluster and the Flink cluster, except, of course, when a query needs to access all of the data in its consumed topics. We considered a cloud-like setting in this evaluation, where the two clusters are connected by high-performance links, but we expect this reduction to yield significant benefits in a geo-distributed setting where communication occurs over a lower-capacity link. Concerning RQ3, by executing projection and selection operations on the broker side, Alloy proxies can, in certain cases, reduce the query’s CPU utilization by more than the additional CPU utilization at the proxy. In other cases, and

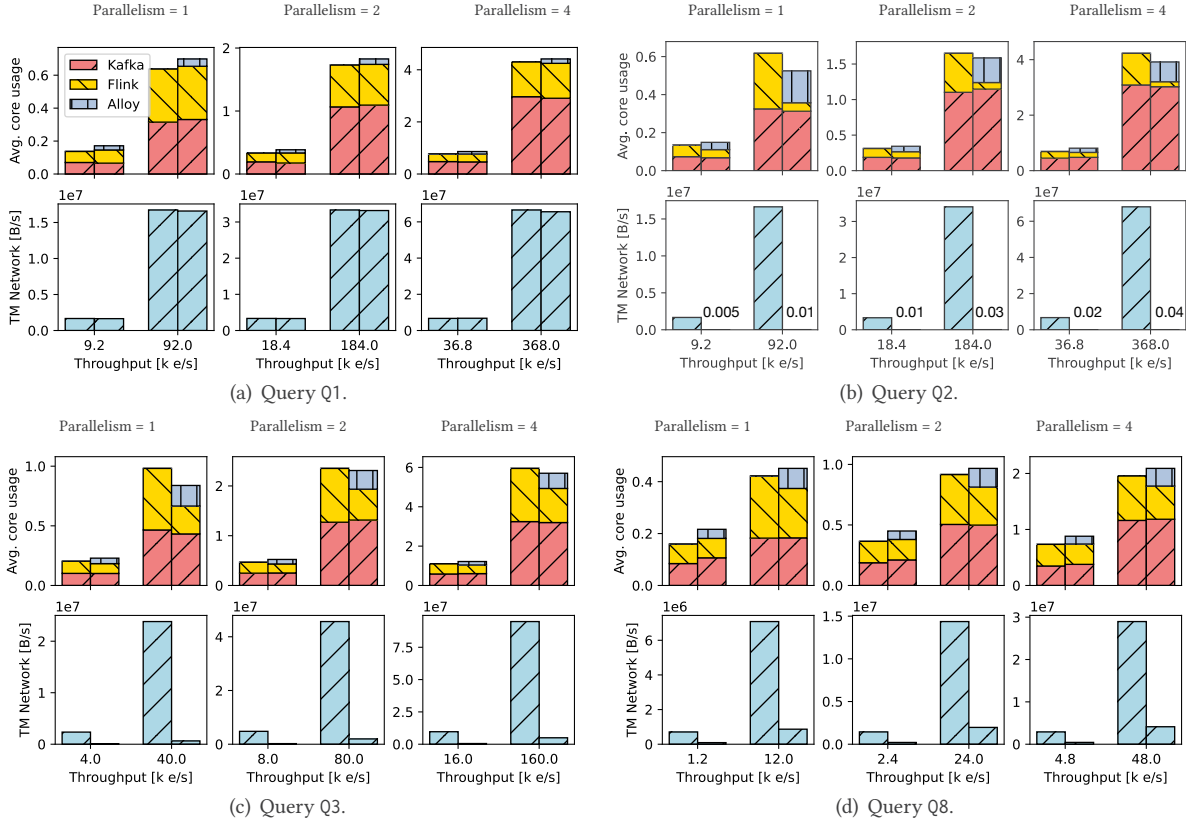


Figure 8: Resource consumption for running queries Q1, Q2, Q3, and Q8 of the Nexmark benchmark. In each subfigure, the three columns correspond to different parallelism levels (1, 2, and 4) for Kafka and the Flink query execution. Each pair of stacked plots shows CPU consumption (top) and network utilization (bottom) for the vanilla coupling of Kafka and Flink (left) and the use of Alloy (right). We consider both a low (first pair of stacked slots) and a high (second pair) effective throughput, given in thousands of events per second.

depending primarily on the input rate, the CPU overhead of the proxy can be slightly higher than the gains. Another point is that part of the CPU load is shifted from the Flink task managers to the Kafka brokers. While Kafka brokers are I/O-bound and typically have lower CPU cores, we think it is reasonable to expect that they have spare CPU capacity that can be used for the proxy operations. Last, we note that the proxies' CPU utilization could likely be reduced by compiling them natively and shipping them along with the Envoy binary, rather than using WebAssembly and the proxy-wasm interface. These gains would, however, come at the cost of reduced flexibility and deployability.

8 Related work

We review related work on data filtering at the source in data center and geo-distributed settings, as well as proxy-based solutions for data storage and processing platforms not otherwise mentioned previously in the paper.

Source filtering in cloud settings. Kafka Streams [27] offers to execute queries directly atop a Kafka infrastructure. KSQL [40] is a SQL interface to Kafka Streams. While useful for simple processing tasks and for creating new topics on disk, e.g., as a subset of existing ones, Kafka Streams has fewer features than Flink: it does not support heterogeneous sources, offers more limited joining operations, and lacks complex window-based analytics. Its performance is also tied to the capacity of existing Kafka brokers, which may also run on CPU-limited, storage-optimized nodes.

Query-able Kafka [33] shares our objectives of enabling data filtering on the Kafka side, in the context of network analytics, where events are large but produced at low or medium velocity. Similar to Alloy, Query-able Kafka offers limited computation, such as filtering and data transformations, on the broker side before forwarding the resulting curated data to the Apache Spark processing engine. Query-able Kafka supports stream transformations, such as per-row aggregation (i.e., within the fields of a single event), that

go beyond Alloy’s current capabilities. It requires, however, deep modifications to Kafka, with the integration of processing capabilities directly in the data reading and buffering logic. This approach leads to poor sustainability. Query-able Kafka’s implementation is indeed bound to an early version of Kafka (0.9.1) and has not been updated in the last 9 years. In contrast, Alloy, by building on the stable Kafka API, has a lower risk of deprecation.

Pre-filtering and pre-computation at the data source are also common techniques in traditional, non-stream databases. Predicate pushdown (and even query pushdown) allows a subset of a relational query to be sent to a storage node while the rest of the query executes elsewhere, saving network bandwidth, e.g., as implemented by SAP HANA [34]. This concept has also been successfully used in batch data science pipelines [70].

Source filtering in distributed environments. Another class of stream processing systems in which processing and filtering data near its source have been extensively studied is that targeting *geo-distributed* settings. A typical example of such a system is the combination of sensors generating real-time data at the edge of the network and cloud-based processing of this data, which may not need to be persisted (i.e., unlike the Kafka model we target). In this class of systems, processing data at the edge reduces the computational load in the cloud and WAN network costs [32].

EdgeWise [37] is a stream processing engine that provides runtime optimization for the limited resources of typical edge-class nodes. It does not, however, provide mechanisms for splitting a query across cloud-class and edge-class nodes. StrIoT [31] does so by providing developers a functional language to express their job. StrIoT can then split operators between the edge and the cloud, performing fission and fusion operations.

Arkian *et al.* study the performance of stream processing applications deployed over fog computing environments, from the perspective of long-distance networking [22] and elastic scaling [23].

NebulaStream [72, 73] is a stream processing engine that targets distributed, heterogeneous, and shared environments. The NebulaStream computation model allows filtering streams near their source, although the system does not allow it in this case to durably store this data for future use, as sending it to a Kafka cluster in the cloud would allow.

Frontier [53] and CSA [64] are two earlier examples of specific stream processing approaches that enable the pre-selection or pre-aggregation of stream data at the edge. Shepherd is another example that focuses on minimizing downtime during reconfiguration for queries deployed across edge and cloud [56].

Proxies for data storage and processing. The use of proxies to transparently extend the capabilities of stream storage and processing systems remains relatively unexplored. Besides the use of Envoy proxies for Kafka we mentioned in Section 2, an interesting example is the work of Kumar *et al.* [47], who propose a proxy approach for Kafka allowing full ordering support for a topic stored across multiple partitions, despite Kafka providing such ordering guarantees only *within* a single partition.

9 Conclusion

We presented Alloy, a proxy-based approach that improves the efficiency of the coupling between a stream storage cluster using Apache Kafka and a stream processing cluster running Apache Flink. Alloy proxies transparently filter events and fields in these events that are of no use to a streaming query, as automatically inferred its control plane. Our evaluation shows the significant gains in network utilization that our approach enables, while keeping the impact on latency very low and keeping overall CPU utilization across clusters on par with, or even lower than, the direct coupling of Kafka and Flink.

The perspectives of this work are numerous. It would be interesting (and challenging) to extend the support of Alloy not only to SQL queries submitted to Flink but also to more general queries, e.g., using the DataStream API or even the general ProcessFunction API, possibly with the help of programmers’ hints at the source code level. As our evaluation has shown, while Alloy can reduce the volume of transferred data when selection and projection operations are applied, CPU usage reduction may depend on the nature of the query and the workload. Integrating the use of Alloy in existing performance and cost models for stream processing [20, 38, 59] would be a promising approach to predict these costs and analyze the tradeoff between network savings and resource utilization. Another perspective would be to integrate filtering capabilities into the network stack and appliances (e.g., SmartNICs and their integrated data-processing-units, DPUs), and save main CPU resources. A final perspective would be to leverage proxies for more advanced data preparation, e.g., to pre-partition data across different Flink nodes to avoid costly re-partitioning and re-balancing operations in the Flink query itself.

Acknowledgments. This work is funded by the Walloon Region (Belgium) Win2Wal project GEPICIAD, by the ARC project “RAINDROP”, and by a gift from Eura Nova. Experiments presented in this paper were carried out using the Grid’5000 testbed, supported by a scientific interest group hosted by Inria and including CNRS, RENATER and several Universities as well as other organizations [25].

References

- [1] 2020. Featuring Apache Kafka in the Netflix Studio and Finance World. <https://www.confluent.io/blog/how-kafka-is-used-by-netflix>.
- [2] 2023. Why Apache Flink is the Industry Gold Standard. <https://www.deltastream.io/why-apache-flink-is-the-industry-gold-standard/>.
- [3] 2024. Envoy: an open source edge and service proxy for cloud-native applications. <https://www.envoyproxy.io>.
- [4] 2024. Modular Open Smart Network. <https://github.com/mosn/mosn?tab=readme-ov-file>.
- [5] 2025. Flink Kubernetes Operator. <https://github.com/apache/flink-kubernetes-operator>.
- [6] 2025. WasmX WebAssembly Modules for Nginx. https://github.com/Kong/nginx_wasm_module.
- [7] 2026. Apache Kafka Connector. <https://nightlies.apache.org/flink/flink-docs-stable/docs/connectors/datastream/kafka/>.
- [8] 2026. Apache Traffic Server (ATS) documentation. <https://docs.trafficserver.apache.org/en/latest/admin-guide/plugins/wasm.en.html>.
- [9] 2026. Azure Event Hubs. <https://learn.microsoft.com/en-us/azure/event-hubs/event-hubs-about>.
- [10] 2026. Diskless Kafka on S3. <https://www.automq.com/>.
- [11] 2026. Istio service mesh. <https://istio.io/>.
- [12] 2026. Kafka Broker filter. https://www.envoyproxy.io/docs/envoy/latest/configuration/listeners/network_filters/kafka_broker_filter.
- [13] 2026. Kafka documentation: Log Compaction. <https://kafka.apache.org/42/design/design/#log-compaction>.
- [14] 2026. Kafka Mesh filter. https://www.envoyproxy.io/docs/envoy/latest/configuration/listeners/network_filters/kafka_mesh_filter.
- [15] 2026. Kafka Streams: documentation. <https://kafka.apache.org/42/streams/>.
- [16] 2026. Realtime Compute for Apache Flink: Configure job resources. <https://www.alibabacloud.com/help/en/flink/realtime-flink/user-guide/configure-deployment-resources>.
- [17] 2026. Running Kafka in Production with Confluent: Hardware Recommendations. <https://docs.confluent.io/platform/current/kafka/deployment.html>.
- [18] 2026. Schema Registry for Confluent Platform. <https://docs.confluent.io/platform/current/schema-registry/index.html>.
- [19] 2026. The Diskless, Kafka Compatible Data Streaming Platform. <https://www.warpstream.com/>.
- [20] Pratyush Agnihotri, Boris Koldehofe, Paul Stiegele, Roman Heinrich, Carsten Binnig, and Manisha Luthra. 2024. ZeroTune: Learned Zero-Shot Cost Models for Parallelism Tuning in Stream Processing. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE Computer Society, 2040–2053.
- [21] Pradeep Ambati, Íñigo Goiri, Felipe Frujeri, Alper Gun, Ke Wang, Brian Dolan, Brian Corell, Sekhar Pasupuleti, Thomas Moscibroda, Sameh Elnikety, et al. 2020. Providing SLOs for Resource-Harvesting VMs in cloud platforms. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.
- [22] Hamidreza Arkian, Guillaume Pierre, Johan Tordsson, and Erik Elmroth. 2020. An experiment-driven performance model of stream processing operators in fog computing environments. In *35th Annual ACM Symposium on Applied Computing (SAC)*.
- [23] HamidReza Arkian, Guillaume Pierre, Johan Tordsson, and Erik Elmroth. 2021. Model-based stream processing auto-scaling in geo-distributed environments. In *International Conference on Computer Communications and Networks (ICCCN)*. IEEE.
- [24] Khaled Arsalane, Guillaume Pierre, and Shadi Ibrahim. 2024. Toward Stream Processing Elasticity in Realistic Geo-Distributed Environments. In *International Conference on Cloud Engineering (IC2E)*. IEEE.
- [25] Daniel Balouek, Alexandra Carpen Amarie, Ghislain Charrier, Frédéric Desprez, Emmanuel Jeannot, Emmanuel Jeanvoine, Adrien Lèbre, David Margery, Nicolas Niclausse, Lucas Nussbaum, Olivier Richard, Christian Pérez, Flavien Quesnel, Cyril Rohr, and Luc Sarzyniec. 2013. Adding Virtualization Capabilities to the Grid'5000 Testbed. In *Cloud Computing and Services Science*, Ivan I. Ivanov, Marten van Sinderen, Frank Leymann, and Tony Shan (Eds.). Communications in Computer and Information Science, Vol. 367. Springer International Publishing, 3–20. doi:10.1007/978-3-319-04519-1_1
- [26] Edmon Begoli, Jesús Camacho-Rodríguez, Julian Hyde, Michael J Mior, and Daniel Lemire. 2018. Apache calcite: A foundational framework for optimized query processing over heterogeneous data sources. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*.
- [27] Bill Bejeck. 2018. *Kafka Streams in Action: Real-time apps and microservices with the Kafka Streams API*. Simon and Schuster.
- [28] Paris Carbone, Stephan Ewen, Gyula Fóra, Seif Haridi, Stefan Richter, and Kostas Tzoumas. 2017. State management in Apache Flink®: consistent stateful distributed stream processing. *Proceedings of the VLDB Endowment* 10, 12 (2017), 1718–1729.
- [29] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. 2015. Apache Flink: Stream and batch processing in a single engine. *The Bulletin of the Technical Committee on Data Engineering* 38, 4 (2015).
- [30] Valeria Cardellini, Francesco Lo Presti, Matteo Nardelli, and Gabriele Russo Russo. 2022. Runtime adaptation of data stream processing systems: The state of the art. *Comput. Surveys* 54, 11s (2022), 1–36.
- [31] Adam Cattermole, Jonathan Dowland, and Paul Watson. 2021. Runtime adaptation of stream processing spanning the cloud and the edge. In *14th IEEE/ACM International Conference on Utility and Cloud Computing Companion (UCC)*. 1–7.
- [32] Marcos Dias De Assuncao, Alexandre da Silva Veith, and Rajkumar Buyya. 2018. Distributed data stream processing and edge computing: A survey on resource elasticity and future directions. *Journal of Network and Computer Applications* 103 (2018), 1–17.
- [33] Eric Falk, Vijay K Gurbani, and Radu State. 2017. Query-able Kafka: An agile data analytics pipeline for mobile wireless networks. *Proceedings of the VLDB Endowment* 10, 12 (2017), 1646–1657.
- [34] Franz Färber, Sang Kyun Cha, Jürgen Primisch, Christof Bornhövd, Stefan Sigg, and Wolfgang Lehner. 2012. SAP HANA database: data management for modern business applications. *ACM Sigmod Record* 40, 4 (2012), 45–51.
- [35] Behrooz Farkiani, Fan Liu, Ke Yang, John DeHart, Jyoti Parwatar, and Patrick Crowley. 2024. Hermes: A General-Purpose Proxy-Enabled Networking Architecture. *arXiv preprint arXiv:2411.13668* (2024).
- [36] Marios Fragkoulis, Paris Carbone, Vasiliki Kalavri, and Asterios Katsifodimos. 2024. A survey on the evolution of stream processing systems. *The VLDB Journal* 33, 2 (2024), 507–541.
- [37] Xinwei Fu, Talha Ghaffar, James C Davis, and Dongyoon Lee. 2019. EdgeWise: A better stream processing engine for the edge. In *2019 USENIX Annual Technical Conference (ATC)*.
- [38] Yuxing Han, Lixiang Chen, Haoyu Wang, Zhanghao Chen, Yifan Zhang, Chengcheng Yang, Kongzhang Hao, and Zhengyi Yang. 2025. Learning from the past: Adaptive parallelism tuning for stream processing systems. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 3535–3548.
- [39] Martin Hirzel, Robert Soulé, Scott Schneider, Buğra Gedik, and Robert Grimm. 2014. A catalog of stream processing optimizations. *ACM Computing Surveys (CSUR)* 46, 4 (2014), 1–34.
- [40] Hojjat Jafarpour, Rohan Desai, and Damian Guy. 2019. KSQL: Streaming SQL Engine for Apache Kafka. In *22nd International Conference on*

- Extending Database Technology, Industry and Applications track (EDBT)*. 524–533.
- [41] Albert Jonathan, Abhishek Chandra, and Jon Weissman. 2020. WASP: Wide-area adaptive stream processing. In *Proceedings of the 21st International Middleware Conference*. 221–235.
 - [42] Vasiliki Kalavri, John Liagouris, Moritz Hoffmann, Desislava Dimitrova, Matthew Forshaw, and Timothy Roscoe. 2018. Three steps is all you need: fast, accurate, automatic scaling decisions for distributed streaming dataflows. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 783–798.
 - [43] David Kjerrumgaard. 2021. *Apache Pulsar in Action*. Simon and Schuster.
 - [44] Konstantin Knauf. 2023. 4 Reasons Why Developers Should Use Apache Flink. <https://thenewstack.io/4-reasons-why-developers-should-use-apache-flink/>.
 - [45] Adam Kotwasinski. 2023. Proxying Kafka with Envoy without changing advertised listeners by using rewrite rules. <https://adam-kotwasinski.medium.com/proxying-kafka-with-envoy-without-changing-advertised-listeners-by-using-rewrite-rules-387792cce066>.
 - [46] Jay Kreps, Neha Narkhede, Jun Rao, et al. 2011. Kafka: A distributed messaging system for log processing. In *Proceedings of the NetDB*, Vol. 11.
 - [47] Shashank Kumar, Aryan Jadon, and Sachin Sharma. 2023. Global Message Ordering using Distributed Kafka Clusters. In *2023 15th International Conference on Innovations in Information Technology (IIT)*. IEEE, 26–31.
 - [48] Wubin Li, Yves Lemieux, Jing Gao, Zhuofeng Zhao, and Yanbo Han. 2019. Service mesh: Challenges, state of the art, and future research opportunities. In *2019 IEEE International Conference on Service-Oriented System Engineering (SOSE)*. IEEE, 122–1225.
 - [49] Liftbridge. [n. d.]. Liftbridge: Lightweight, fault-tolerant message streams. <https://liftbridge.io>.
 - [50] Habib Mostafaei and Shafi Afridi. 2023. SDN-enabled resource provisioning framework for geo-distributed streaming analytics. *ACM Transactions on Internet Technology* 23, 1 (2023), 1–21.
 - [51] Habib Mostafaei, Georgios Smaragdakis, Thomas Zinner, and Anja Feldmann. 2022. Delay-resistant geo-distributed analytics. *IEEE Transactions on Network and Service Management* 19, 4 (2022), 4734–4749.
 - [52] NATS. [n. d.]. JetStream: NATS documentation. <https://docs.nats.io/nats-concepts/jetstream>.
 - [53] Dan O’Keeffe, Theodoros Salonidis, and Peter Pietzuch. 2018. Frontier: Resilient edge processing for the internet of things. *Proceedings of the VLDB Endowment* 11, 10 (2018), 1178–1191.
 - [54] proxy wasm. 2021. WebAssembly for Proxies (ABI specification). <https://github.com/proxy-wasm/spec>
 - [55] RabbitMQ. [n. d.]. RabbitMQ: One broker to queue them all. <https://www.rabbitmq.com/>.
 - [56] Brian Ramprasad, Pritish Mishra, Myles Thiessen, Hongkai Chen, Alexandre da Silva Veith, Moshe Gabel, Oana Balmau, Abelard Chow, and Eyal de Lara. 2022. Shepherd: Seamless stream processing on the edge. In *7th IEEE/ACM Symposium on Edge Computing (SEC)*.
 - [57] Redpanda. [n. d.]. RedPanda: The streaming data platform for developers. <https://redpanda.com/>.
 - [58] Henriette Röger and Ruben Mayer. 2019. A comprehensive survey on parallelization and elasticity in stream processing. *ACM Computing Surveys (CSUR)* 52, 2 (2019), 1–37.
 - [59] Guillaume Rosinosky, Donatien Schmitz, and Etienne Rivière. 2024. StreamBed: capacity planning for stream processing. In *18th ACM International Conference on Distributed and Event-based Systems (DEBS)*.
 - [60] Guillaume Rosinosky, Donatien Schmitz, and Etienne Rivière. 2026. **Source code of Alloy and reproducibility instructions**. <https://zenodo.org/records/19498332>.
 - [61] Harshit Saokar, Soteris Demetriou, Nick Magerko, Max Kontorovich, Josh Kirstein, Margot Leibold, Dimitrios Skarlatos, Hitesh Khandelwal, and Chunqiang Tang. 2023. ServiceRouter: Hyperscale and Minimal Cost Service Mesh at Meta. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 969–985.
 - [62] Donatien Schmitz, Luc Berrewaerts, Guillaume Rosinosky, Sabri Skhiri, and Etienne Rivière. 2025. A Tale of Many Streams: Characterizing a Hybrid Batch-Stream Production Workload in Digazu, a Data Lake supported by Apache Kafka and Flink. In *19th ACM International Conference on Distributed and Event-based Systems (DEBS)*. 188–198.
 - [63] Donatien Schmitz, Guillaume Rosinosky, and Etienne Rivière. 2025. Justin: Hybrid CPU/Memory Elastic Scaling for Distributed Stream Processing. In *IFIP International Conference on Distributed Applications and Interoperable Systems (DAIS)*. Springer, 102–118.
 - [64] Zhitao Shen, Vikram Kumaran, Michael J Franklin, Sailesh Krishnamurthy, Amit Bhat, Madhu Kumar, Robert Lerche, and Kim Macpherson. 2015. CSA: Streaming Engine for Internet of Things. *IEEE Data Eng. Bull.* 38, 4 (2015), 39–50.
 - [65] Ankit Toshniwal, Siddarth Taneja, Amit Shukla, Karthik Ramasamy, Jignesh M Patel, Sanjeev Kulkarni, Jason Jackson, Krishna Gade, Maosong Fu, Jake Donham, et al. 2014. Storm at Twitter. In *Proceedings of the 2014 ACM SIGMOD international conference on Management of data*. 147–156.
 - [66] Pete Tucker, Kristin Tufte, Vassilis Papadimos, and David Maier. 2010. NEXMark—a benchmark for queries over data streams. Technical Report. OGI School of Science & Engineering at OHSU.
 - [67] Juliane Verwiebe, Philipp M Grulich, Jonas Traub, and Volker Markl. 2023. Survey of window types for aggregation in stream processing systems. *The VLDB Journal* 32, 5 (2023), 985–1011.
 - [68] Yingjun Wu. 2023. Stream Processing: Is SQL Good Enough? <https://medium.com/better-programming/stream-processing-is-sql-good-enough-559356a72df8>.
 - [69] Le Xu, Shivaram Venkataraman, Indranil Gupta, Luo Mai, and Rahul Potharaju. 2021. Move fast and meet deadlines: Fine-grained real-time stream processing with Cameo. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*. 389–405.
 - [70] Cong Yan, Yin Lin, and Yeye He. 2023. Predicate pushdown for data science pipelines. *Proceedings of the ACM on Management of Data* 1, 2 (2023), 1–28.
 - [71] Matei Zaharia, Tathagata Das, Haoyuan Li, Timothy Hunter, Scott Shenker, and Ion Stoica. 2013. Discretized streams: Fault-tolerant streaming computation at scale. In *Proceedings of the twenty-fourth ACM symposium on operating systems principles*. 423–438.
 - [72] Steffen Zeuch, Ankit Chaudhary, Bonaventura Del Monte, Haralampos Gavrilidis, Dimitrios Giouroukis, Philipp M Grulich, Sebastian Bress, Jonas Traub, and Volker Markl. 2020. The NebulaStream Platform: Data and Application Management for the Internet of Things. In *Conference on Innovative Data Systems Research (CIDR)*.
 - [73] Steffen Zeuch, Ankit Chaudhary, Viktor Rosenfeld, Taha Tekdogan, Adrian Michalke, Matthias Gördel, Ariane Ziehn, and Volker Markl. 2024. Using and Enhancing NebulaStream-A Tutorial. In *18th ACM International Conference on Distributed and Event-based Systems (DEBS)*.